

Enhancing Student Learning Through Automated Assistance

Supervised by : Mr.J.R.K.C.Jayakody

202069 - M.A.Y.Iwanthi

202151 - A.M.K.S.Sandunika

202162 - D.G.P.M.Shantha

202260 - V.P.Y.Kawindi

202262 - H.G.G.K.Madulakshan



Table of contents

01 INTRODUCTION

02 DATASET

03 MODEL

01. INTRODUCTION

This project aims to create an automated Q&A System to assist students in finding answers to questions from course materials or textbooks, enhancing learning by providing instant, accurate responses.



Why Fine-tuning a pre-trained model is preferred over creating a model from scratch ?



1. Leverage Pre-trained Knowledge

- **Pre-trained Models Examples:**
BERT, GPT, RoBERTa trained on massive datasets (e.g., Wikipedia, books, web data).
- **What They Already Know:**
 - Word embeddings: Contextual relationships between words.
 - General linguistic features: Syntax, grammar, semantics.
- **Advantages of Fine-Tuning:**
 - Adapts pre-trained knowledge to specific tasks (e.g., Question Answering).
 - Saves time and reduces computational effort.



2. Efficiency

- **Time-Saving:**
 - Avoids weeks/months of training from scratch.
 - No need to train billions of parameters from random initialization.
- **Cost-Effective:**
 - Reduces reliance on expensive GPUs/TPUs.
 - Makes advanced models accessible for developers with limited budgets



3. Smaller Dataset Requirements

Training From Scratch:

- Requires a massive labeled dataset to learn patterns effectively.

Fine-Tuning Benefits:

- Needs a smaller labeled dataset (e.g., SQuAD v2).
- Adapts a pre-trained model to specific tasks/domains with just a few thousand examples.
- Utilizes the model's pre-learned general language understanding.



4. Avoiding Overfitting

Challenges of Training from Scratch:

- Small datasets risk overfitting, where the model memorizes training data but fails on unseen data.

Advantages of Pre-Trained Models:

- Provide a strong starting point with general knowledge.
- Help reduce overfitting during fine-tuning.



5. Proven Architectures

Proven Architectures:

- Pre-trained models like BERT, RoBERTa, and GPT are optimized for language tasks.

Challenges of Starting from Scratch:

- Requires designing and validating new architectures.
- Involves extensive hyperparameter tuning and experimentation.

Fine-Tuning Benefit:

- Avoids architecture experimentation and focuses on task-specific adaptation.



6. Better Performance

Better Performance:

- Fine-tuned models often outperform models trained from scratch.

Example:

- BERT fine-tuned on SQuAD achieved state-of-the-art results in Question Answering.

Generalization Advantage:

- Pre-trained models capture universal language patterns, leading to better generalization even with sufficient data.



7. Scalability

Scalable Approach:

- A single base model can be fine-tuned for multiple tasks (e.g., QA, text classification, summarization).

Reusable and Modular:

- Fine-tuning eliminates the need for retraining from scratch, making it efficient and adaptable.



8. Real-World Applicability

Limited Resources for Training From Scratch:

- Most real-world scenarios lack the resources or labeled data to train a model from scratch.

Pre-Trained Models Advantage:

- Designed for easy fine-tuning, making them practical for developers and researchers with minimal effort.

Why Choose BERT for SQuAD QA

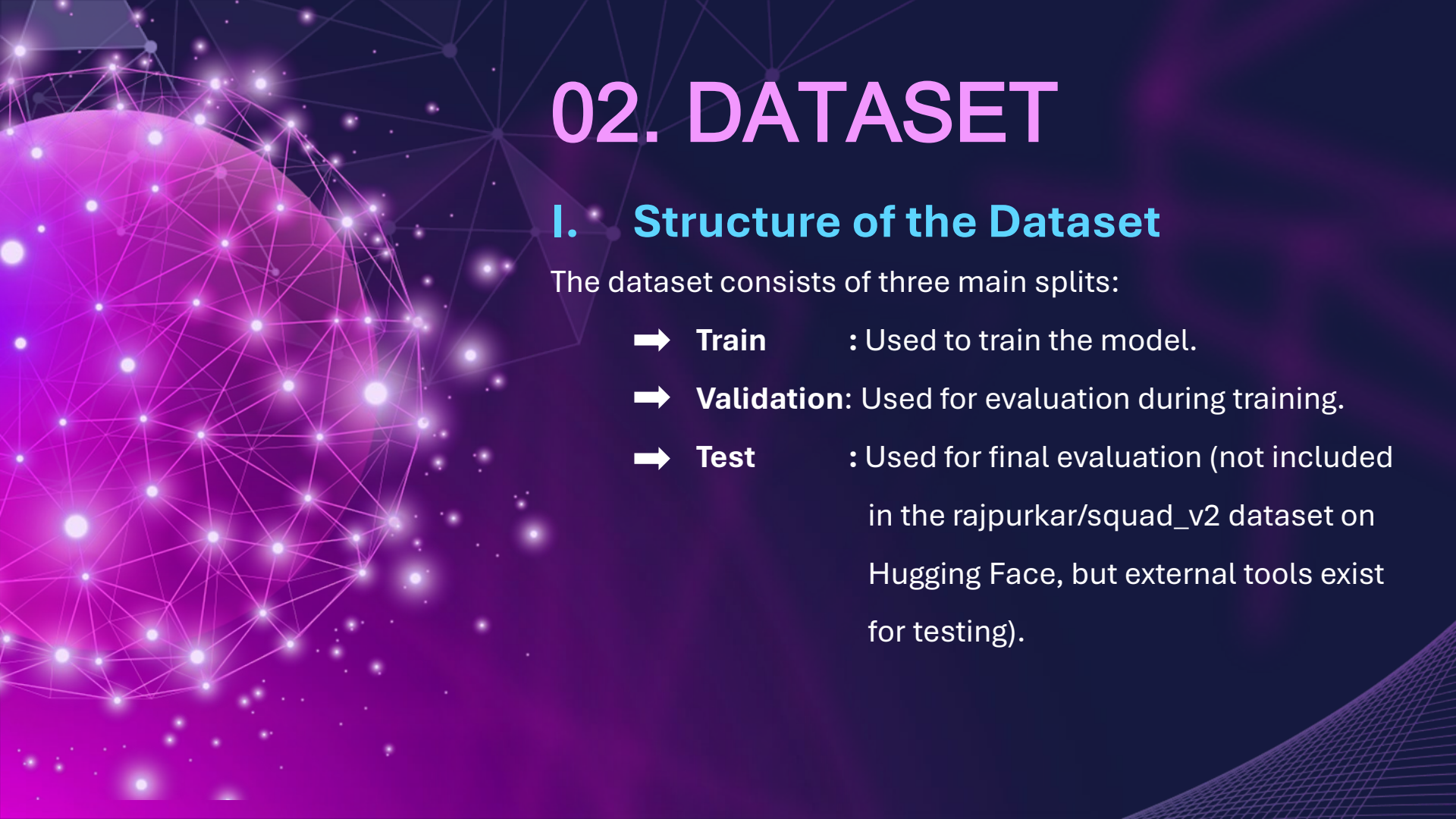
BERT's Strengths for QA Tasks:

➤ Bidirectional Context Understanding:

- BERT processes text bidirectionally (left-to-right and right-to-left).
- This allows BERT to fully understand the context of a word in a sentence.
- Example: In "The bank is on the river," BERT can differentiate between a financial institution and a riverbank.
- Crucial for QA tasks, as precise context understanding is key for correct answer extraction.

➤ Pre-trained on QA-Specific Tasks:

- BERT has been fine-tuned on QA datasets.
- The "start token" and "end token" method fits well with extractive QA formats.



02. DATASET

I. Structure of the Dataset

The dataset consists of three main splits:

- ➔ **Train** : Used to train the model.
- ➔ **Validation**: Used for evaluation during training.
- ➔ **Test** : Used for final evaluation (not included in the rajpurkar/squad_v2 dataset on Hugging Face, but external tools exist for testing).

II. How is the Dataset Used?

1. Training:

- Models are trained to predict:
 - Start and end token positions for the answer (if it exists).
 - A **special "no-answer" probability** for unanswerable questions.

2. Evaluation:

- During validation or testing, models are evaluated using:
 - Exact Match (EM):** Measures the percentage of predictions that exactly match any one of the ground-truth answers.
 - F1 Score:** Measures the overlap between the prediction and ground-truth answers as a harmonic mean of precision and recall.

```
{  
  "id": "56ddde6b9a695914005b9628",  
  "title": "Normans",  
  "context": "The Normans were the  
people who lived in Normandy, a  
region in northern France...",  
  "question": "Who were the  
Normans?",  
  "answers": {  
    "text": ["The people who lived in  
Normandy"],  
    "answer_start": [4]  
  }  
}
```

III. Dataset Statistics

- **Number of Examples:**
 - Training set: ~130,000 examples (answerable + unanswerable).
 - Validation set: ~12,000 examples.
- **Answerable vs Unanswerable Questions:**
 - Around 35-40% of the questions are unanswerable, making it a challenging dataset.



03. MODEL

Comprehensive implementation for fine-tuning a BERT model for Question Answering (QA) using the SQuAD v2 dataset.

1. Loading and Inspecting the Dataset

```
from datasets import load_dataset  
ds = load_dataset("rajpurkar/squad_v2")
```

What is happening?

The **SQuAD v2 dataset** (Stanford Question Answering Dataset) is loaded using the **datasets** library. It contains:

Questions: A question asked about a given context.

Context: A passage of text from which the answer is derived.

Answers: The correct answer(s) to the question. (SQuAD v2 includes unanswerable questions.)

```
print(ds['train'][0])
```

Prints the first sample of the training set, showing the structure: question, context, answers.

2. Preprocessing the Dataset

```
from transformers import BertTokenizerFast  
tokenizer = BertTokenizerFast.from_pretrained("bert-base-uncased")
```

What is happening?

A **fast tokenizer** for BERT is initialized, which will tokenize the context and question text for the model.

Preprocessing Function

```
def preprocess_data(examples):  
    inputs = tokenizer(  
        examples['question'],  
        examples['context'],  
        truncation="only_second",  
        max_length=384,  
        stride=128,  
        padding="max_length",  
        return_overflowing_tokens=True,  
        return_offsets_mapping=True  
    )
```

Inputs tokenization:

- **truncation="only_second"**: Truncates the context if the total length exceeds the maximum token length (384).
- **stride=128**: If the context is too long, it creates overlapping chunks with a stride of 128 tokens.
- **return_overflowing_tokens=True**: Ensures chunks of long contexts are returned.
- **return_offsets_mapping=True**: Provides the mapping between characters in the original context and token indices.

Mapping Answers to Tokens

```
offset_mapping = inputs.pop("offset_mapping")
answers = examples["answers"]
```

Extracts the **character offsets** of tokens and the ground-truth **answer** positions.

```
for i, offsets in enumerate(offset_mapping):
    if i >= len(answers) or "answer_start" not in answers[i] or
len(answers[i]["answer_start"]) == 0:
        start_positions.append(0)
        end_positions.append(0)
```

- If the question is unanswerable, **start** and **end positions** are set to 0.

```
else:
    start_char = answers[i]["answer_start"][0]
    end_char = start_char + len(answers[i]["text"][0])
```

- Retrieves the **start and end characters** of the answer in the context.

```
for idx, (start, end) in enumerate(offsets):
    if start <= start_char < end:
        token_start_index = idx
    if start < end_char <= end:
        token_end_index = idx
    break
```

- Maps the **start and end characters** of the answer to token indices in the tokenized context.

```
inputs["start_positions"] = start_positions
inputs["end_positions"] = end_positions
return inputs
```

- Adds start_positions and end_positions for training.

3. Fine-tuning BERT

```
from transformers import BertForQuestionAnswering, TrainingArguments, Trainer,
DataCollatorWithPadding
```

- **BertForQuestionAnswering:** A pre-trained BERT model designed for QA tasks.
- **DataCollatorWithPadding:** Dynamically pads the input to the maximum length in a batch.

Training Arguments

```
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    weight_decay=0.01,
)
```

Specifies the training configuration:

- **output_dir:** Directory to save model checkpoints.
- **evaluation_strategy="epoch":** Evaluates the model after each epoch.
- **learning_rate:** Optimizer learning rate.
- **Batch size:** 8 for both training and evaluation.
- **num_train_epochs:** 3 training epochs.
- **weight_decay:** Regularization for the optimizer.

Trainer

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_ds["train"],  
    eval_dataset=tokenized_ds["validation"],  
    data_collator=data_collator,  
)
```

Trainer simplifies training and evaluation.

train_dataset: Preprocessed training dataset.

eval_dataset: Validation dataset.

```
trainer.train()
```

Starts the fine-tuning process.

4. Evaluating Checkpoints

```
checkpoints = [os.path.join(checkpoints_dir, d) for d in
os.listdir(checkpoints_dir) if d.startswith("checkpoint")]
```

- Lists all model checkpoints saved during training.

```
qa_pipeline = pipeline("question-answering", model=model, tokenizer=tokenizer,
device=device)
```

- Initializes a **QA pipeline** for inference using the saved checkpoint.

```
result = qa_pipeline(question=question, context=context)
```

- Evaluates the pipeline on a test question (question="What is the capital of France?") and context.

Selecting the Best Checkpoint

```
if result['score'] > best_score:
    best_score = result['score']
    best_checkpoint = checkpoint
```

Tracks the best-performing checkpoint based on **score**

5. Saving the Best Model

```
tokenizer.save_pretrained(final_model_path)
model.save_pretrained(final_model_path)
```

Saves the best checkpoint to a permanent directory for later use.

6. Inference with the Fine-tuned Model

```
qa_pipeline = pipeline("question-answering", model=model, tokenizer=tokenizer, device=0)
result = qa_pipeline(question=question, context=context)
```

- Uses the fine-tuned model to answer questions such as:

Question: "What is the banker's algorithm?"

Context: "The Banker's algorithm is a resource allocation and deadlock avoidance algorithm..."

- Prints:
- Answer: "The Banker's algorithm is a resource allocation and deadlock avoidance algorithm"

Q & A...?





Thank You !